Design, Deployment, and Cloud-Native Enhancement of a Three-Tier Web Application

Cloud Computing - (SE-490) - Complex Engineering Activity (CEA)

Department of Software Engineering

As a Cloud/DevOps Engineer, you are required to design, containerize, and deploy a three-tier web application on a cloud platform of your choice, and to build a CI/CD pipeline that automates the build and deployment process.

System Requirements

Students must design and deploy a solution satisfying all of the following requirements.

1. Network Layer

- A Virtual Network / VPC with an appropriate CIDR/address range
- Public subnet(s) for internet-facing components
- Private subnet(s) for internal/backend components
- Internet connectivity (Internet Gateway / equivalent) for public subnets
- Appropriate route tables / routing rules
- NAT gateway or equivalent, if private-subnet resources require outbound internet access

2. Web Tier

Deploy the Web Tier as a Docker container (e.g., Nginx, Apache, or a front-end application), built from a well-structured Dockerfile. The Web Tier must be publicly accessible via a web browser, typically placed in the public subnet or behind a load balancer.

3. Application Tier

Deploy the Application Tier as a Docker container performing application/business logic (e.g., PHP, Python/Flask or Django, Node.js, Java/Spring Boot). Use a Dockerfile and, if the Web and Application tiers run together on the same host, a docker-compose.yml for local multi-service orchestration. The Application Tier must communicate securely with the Database Tier and should generally reside in a private subnet, reachable only from the Web Tier.

4. Database Tier

Deploy a managed database service (e.g., Amazon RDS, Azure Database, Google Cloud SQL) or a properly secured self-managed database instance. Supported engines may include

MySQL, PostgreSQL, or MariaDB. The database must be accessible only through controlled network access — never directly exposed to the public internet.

5. Container Registry

Push the built Docker image(s) for the Web and/or Application tier to a container registry — e.g., GitHub Container Registry (GHCR), Docker Hub, Amazon ECR, Azure Container Registry, or Google Artifact Registry.

6. CI/CD Pipeline

Build a CI/CD pipeline that automatically builds, tests, and deploys the application on every code change — e.g., using GitHub Actions, GitLab CI/CD, Azure DevOps Pipelines, Jenkins, or CircleCI. The pipeline must, at minimum: build the Docker image, push it to the registry, and deploy/update the running application (e.g., by pulling the new image onto the target compute instance or updating a Kubernetes deployment).

7. Load Balancing and Scalability

Students should implement, where appropriate to their chosen platform:

- A load balancer (e.g., Application Load Balancer, Azure Load Balancer, GCP Load Balancing, or an Nginx/HAProxy-based equivalent)
- Target groups / backend pools
- Auto Scaling Groups or equivalent horizontal scaling mechanism

8. Security

Implement and justify suitable security controls, including:

- Security groups / firewall rules per tier
- Network segmentation between public and private subnets
- Principle of least privilege for inter-tier communication
- Secrets/credentials management (avoid hardcoding database credentials, API keys, etc.), including secrets used within the CI/CD pipeline

9. Additional Cloud-Native Enhancements (Choose at least one)

- Configure object storage (e.g., Amazon S3, Azure Blob Storage, Google Cloud Storage) to store static assets, uploaded files, or application backups.
- Provision and attach persistent block/file storage (e.g., Amazon EBS/EFS, Azure Disk/Files, GCP Persistent Disk/Filestore) where appropriate.
- Deploy the containerized application using Kubernetes (a managed service such as EKS/AKS/GKE, or a local cluster such as Minikube/Kind), including Kubernetes manifests (Deployment, Service, Ingress, etc.).

- Implement centralized logging and/or monitoring (e.g., CloudWatch, Azure Monitor, Google Cloud Operations, Prometheus/Grafana).
- Implement infrastructure-as-code for the deployment (e.g., Terraform, CloudFormation, Bicep, Pulumi).
- Estimate the monthly infrastructure cost using the relevant provider's pricing calculator and propose at least two cost optimization measures.

10. Non-Functional Requirements (NFR) Justification

A cloud-native deployment is judged not only by whether it works, but by whether it satisfies the quality attributes that matter for production systems. Students must explicitly justify, with reference to their own architecture and implementation choices, how their solution addresses each of the following NFRs:

- **Scalability** — how the design accommodates growth in load (e.g., horizontal scaling via Auto Scaling Groups, statelessness of application containers, load balancer distribution).
- **Availability / Reliability** — how the system avoids single points of failure (e.g., multi-AZ/region placement, managed database failover, redundant compute instances).
- **Elasticity** — how resources scale up and down automatically in response to demand, rather than being statically provisioned.
- **Security** — how confidentiality, integrity, and controlled access are enforced across tiers, the network, and the CI/CD pipeline itself.
- **Maintainability / Deployability** — how containerization and the CI/CD pipeline reduce manual effort and risk when releasing new versions.
- **Performance / Latency** — how the architecture (e.g., load balancing, caching, resource sizing, subnet/region placement) supports acceptable response times under expected load.
- **Cost-Efficiency** — how the chosen services and scaling strategy balance performance against running cost.
- **Portability** — how containerization reduces coupling to a specific cloud provider and eases migration.

You are not required to implement every NFR to an advanced degree, but must clearly state which design decisions support which NFRs, and honestly identify any NFRs your current solution does not fully address, along with how they could be improved.

Required Deliverables

Each student/group must submit:

1. **Architecture Diagram** A clearly labeled diagram showing all cloud services used, network design, connectivity between components, the container/CI-CD flow, and security boundaries.

2. **Deployment Evidence (Screenshots)**
    - Network/VPC configuration and subnets
    - Compute instances / container hosts
    - Security groups / firewall rules
    - Load balancer configuration
    - Database instance
    - Docker image build and registry listing
    - CI/CD pipeline run (successful build and deploy)
    - The running application, accessed via a browser
3. **Source Files**
    - Dockerfile (and docker-compose.yml if used)
    - Kubernetes manifests (if applicable)
    - CI/CD pipeline definition file (e.g., deploy.yml for GitHub Actions)
    - Link to the source code repository
4. **Technical Report (4–6 pages)** The report should explain:
    - Chosen cloud provider and justification
    - Overall architecture and design decisions
    - Services used and their role in each tier
    - Containerization and CI/CD pipeline design
    - Security considerations
    - Testing results demonstrating successful automated deployment
    - **NFR justification** — a dedicated section mapping specific design/implementation decisions to the non-functional requirements listed in System Requirements item 10, including any known limitations